Comprehensive Architectural Blueprint and Database Schema Modernization for GSMSharing.com

Executive Summary

This report presents a rigorous architectural analysis and database schema design for the modernization of GSMSharing.com. The primary objective is to transition the platform from a legacy, script-based architecture into a high-performance, scalable ecosystem built on the ASP.NET Core 10 stack, enhanced by Python-driven computational intelligence. The proposed solution addresses the complex convergence of three distinct digital domains: a detailed technical content repository (comparable to GSMArena.com), a bustling community forum, and a robust multi-vendor marketplace for electronics and software tools.

The analysis draws deeply from the operational success of industry leaders like GSMArena, leveraging their data structuring principles while integrating the transactional complexity required for modern e-commerce. A key focus is placed on resolving the historic tension between rigid relational schemas and the rapidly evolving nature of mobile device specifications. To this end, the report details a hybrid Relational-Document data model, utilizing the advanced JSON capabilities of modern SQL engines (PostgreSQL or SQL Server) which are fully compatible with Entity Framework Core 10.

Furthermore, this document outlines the integration of Python as an asynchronous content enhancement layer, the implementation of a high-precision affiliate tracking system, and a granular permission system for a gamified community experience. The legacy context—defined by an "old script.sql" and "modified gsmsharing_core"—dictates a "Strangler Fig" migration strategy, ensuring business continuity while systematically eliminating technical debt. This blueprint serves as the definitive technical guide for the engineering team to execute this transformation.

---

1. Domain Analysis and Competitive Benchmarking

To engineer a superior database schema for GSMSharing, it is imperative to first deconstruct the existing operational models of the target domain. This involves a dual analysis: dissecting the industry standard for information density, GSMArena.com, and evaluating the current commercial and functional reality of GSMSharing.com.

1.1 Deconstructing the GSMArena Data Model

GSMArena represents the gold standard for mobile phone specifications. An analysis of their "Phone Finder" and "Compare" tools reveals a data model that is both rigid in its categorization and expansive in its attribute depth. Their system manages thousands of devices, each characterized by over 30 distinct specification categories ranging from basic physical dimensions to complex network frequency support.[1]

The GSMArena model is predicated on distinct data types:

- **Static Manufacturer Data:** This includes immutable specifications provided by OEMs (Original Equipment Manufacturers) such as screen resolution (e.g., "1440 x 3120 pixels"), chipset architecture (e.g., "Snapdragon 8 Gen 3"), and battery capacity.
- **Dynamic Proprietary Metrics:** A critical differentiator is their proprietary testing data. The "Battery Life Table" and "Loudspeaker Tests" are not manufacturer specs but platform-generated data points. These require a schema that separates *claimed* specs from *verified* performance metrics.[1]
- **Relational Interconnectivity:** The "Coverage" map implies a many-to-many relationship between devices and cellular bands.[1] A single marketing name (e.g., "iPhone 15") maps to multiple hardware SKUs (A2846, A2848), each supporting a different subset of global frequencies (n1, n3, n5, mmWave). A flat table structure fails here; a normalized relational approach is mandatory to support features like "Will this phone work in my country?"

Implications for GSMSharing:
The new schema must adopt this granularity. It cannot simply store a text blob for "Network." It must model NetworkBands, DeviceVariants, and RegionCompatibility as distinct entities to enable the advanced filtering and SEO optimization required by the user.3

1.2 The GSMSharing Hybrid Proposition

While GSMArena focuses on information, GSMSharing integrates direct commerce and digital distribution. The analysis of GSMSharing.com reveals a broader functional scope that includes "Websites for sale," "Active Topics" in forums, and a diverse product catalog ranging from high-end foldables to service tools like the "Samsung Z3X Pro Box".[4]

This hybrid nature introduces conflicting database requirements:

1. **Read-Heavy Optimization:** Like GSMArena, the specification pages require massive read throughput. Users browsing the "Samsung S25 Ultra vs S24 Ultra" article need sub-millisecond data retrieval.[4]
2. **Write-Heavy Throughput:** The forum and marketplace components are transactional. "Active Topics" and "Buy Now" actions generate significant write locks.
3. **Binary/Blob Management:** The platform's unique selling proposition—"download flashing files"—requires a schema optimized for large file metadata management, differentiating it from GSMArena's pure text/image focus.[4]

1.3 Strategic Migration from Legacy SQL

The user's reference to "old script.sql" and "modified gsmsharing_core" signals the presence of a legacy codebase, likely a LAMP stack (Linux, Apache, MySQL, PHP) application or an older ASP.NET WebForms system. These legacy schemas typically suffer from:

- **MyISAM Table Locking:** Causing performance bottlenecks in forums.
- **Inconsistent Naming Conventions:** Mixing snake_case and PascalCase.
- **Denormalization:** Storing serialized PHP arrays in text columns, making data unqueryable.

The modernization strategy must treat the "old script.sql" not as a blueprint to copy, but as a data source for an Extract-Transform-Load (ETL) pipeline. The new schema must be strictly normalized where appropriate (Identity, Orders) and leverage modern JSONB columns for flexible data (Specs), completely decoupling the new architecture from the limitations of the "modified gsmsharing_core."

---

2. Core Architecture and Technology Stack Integration

The selected stack—ASP.NET Core 10 and Python—dictates a specific architectural approach. ASP.NET Core 10 (anticipated) builds upon the performance improvements of.NET 8/9, offering native AOT (Ahead-of-Time) compilation and enhanced Entity Framework Core capabilities.

2.1 The Modular Monolith Architecture

Given the requirement to support distinct domains (Marketplace, Forum, Specs, Files) within a single "gsmsharing.com" ecosystem, a **Modular Monolith** is the optimal architectural pattern. Unlike microservices, which introduce network latency and distributed transaction complexity, a modular monolith shares a single database instance but enforces logical separation via database *Schemas*.

Database Schema Segmentation:

- Identity: User accounts, profiles, auth tokens.
- Catalog: Device specifications, brands, series (The "GSMArena" clone).
- Commerce: Orders, products, vendors, shipping.
- Community: Forums, threads, posts, reputation.
- Content: News articles, SEO metadata, tags.
- Analytics: Affiliate tracking, click logs.

This separation allows for easier future splitting. If the Analytics tables grow too large, that specific schema can be migrated to a dedicated server or a time-series database without refactoring the entire application.

2.2 Python as the Computational Intelligence Layer

The user specifies Python for "content enhancement." In a.NET ecosystem, this implies an interoperability layer. We should not use Python to serve HTTP requests directly to users (which ASP.NET Core handles faster). Instead, Python serves as a background worker service.

Integration Schema:
The database will act as the communication bus.

1. **Job Queue Table:** ASP.NET Core inserts a record into Sys_PythonJobs when a new device is added.
2. **Processing:** A Python daemon (using SQLAlchemy) polls this table.
3. **Execution:** Python executes complex tasks:
    - *Natural Language Processing (NLP):* Generating "Verdict" summaries from raw specs.
    - *Data Scraper:* Fetching missing bands or battery specs from external APIs.[5]
    - *Image Optimization:* Analyzing uploaded product images for quality.
4. **Result Write-back:** Python updates the Catalog.DeviceEnrichments table.

2.3 Legacy Data Ingestion Strategy

To handle the "old script.sql," the new schema includes a temporary Legacy schema.

- **Phase 1:** Restore old script.sql into the Legacy schema of the new database.
- **Phase 2:** Use Entity Framework Core seed scripts to map Legacy.users to Identity.AspNetUsers.
- **Phase 3:** Map Legacy.products to Commerce.Listings.
- **Phase 4:** Once verification is complete, drop the Legacy schema.

---

3. The Product Specification Engine (The "GSM" Core)

This is the intellectual core of the platform. The schema must handle the extreme variability of electronics specifications. A phone has a "Screen Size," but a "Z3X Pro Box" (hardware tool) has "Supported Models" and "Cable Type." A traditional Relational model (one column per attribute) fails here because the column count would exceed database limits.

3.1 The Failure of EAV and the Rise of JSONB

Historically, platforms used the Entity-Attribute-Value (EAV) model (Tables: Products, Attributes, ProductAttributeValues). While flexible, EAV is disastrous for performance. To compare three phones, the database must perform dozens of self-joins, killing query speed.6
Solution: The Hybrid Relational-JSONB Model. We use relational columns for fields that are sorted or filtered globally (Price, Name, Release Date) and a binary JSON column (JSONB in PostgreSQL, JSON in SQL Server) for the variable specifications. EF Core 10 allows LINQ queries directly into these JSON documents, offering the best of both SQL and NoSQL worlds.8

3.2 Core Catalog Schema

This schema manages the "source of truth" for devices, mirroring the depth of GSMArena.

Table: Catalog.Brands

Column	Type	Description
Id	INT	Primary Key.
Name	NVARCHAR(100)	"Samsung", "Apple", "Z3X". Indexed, Unique.
Slug	VARCHAR(100)	URL-safe version: "samsung".
LogoUrl	VARCHAR(500)	CDN path to brand logo.
WebsiteUrl	VARCHAR(500)	Official manufacturer site.
IsPremium	BIT	Highlights top partners or popular brands.

Table: Catalog.Devices
The master record for a generic product (e.g., "iPhone 15 Pro").

Column	Type	Description
Id	UUID	Primary Key.
BrandId	INT	Foreign Key to Brands.
Name	NVARCHAR(200)	"Galaxy S24 Ultra".
Slug	VARCHAR(250)	"samsung-galaxy-s24-ultra ". Unique Index.
ReleaseStatus	TINYINT	0=Rumored, 1=Announced, 2=Released,

		3=Discontinued.
AnnouncedDate	DATE	For "Newest Devices" sorting.
ThumbnailUrl	VARCHAR(500)	Main catalog image.
SearchVector	TSVECTOR	(Postgres specific) Pre-computed text search vector.
Specifications	JSONB	**Crucial:** Stores all dynamic technical data.

Table: Catalog.DeviceVariants
GSMArena data shows that a single "Device" has multiple SKUs. The Devices table holds the concept; Variants holds the hardware.

Column	**Type**	**Description**
Id	UUID	PK.
DeviceId	UUID	FK to Devices.
ModelCode	VARCHAR(50)	e.g., "SM-S928B/DS".
Region	VARCHAR(50)	"EU", "USA", "Global".
RamBytes	BIGINT	Standardized for comparison.
StorageBytes	BIGINT	Standardized for comparison.
NetworkBands	JSONB	Array of supported bands specific to this variant.

3.3 The JSONB Specification Structure

The Specifications column in Catalog.Devices allows for deep nesting, mimicking the categorization found on GSMArena (Display, Platform, Camera).

JSON

```
{
  "display": {
    "type": "Dynamic AMOLED 2X",
    "size_inches": 6.8,
    "resolution_pixels": "1440 x 3120",
    "protection": "Corning Gorilla Armor",
    "features":
  },
  "platform": {
    "os": "Android 14, One UI 6.1",
    "chipset": "Qualcomm SM8650-AC Snapdragon 8 Gen 3",
    "cpu_cores": 8,
    "gpu": "Adreno 750"
  },
  "main_camera": {
    "modules":,
    "video_capabilities": ["8K@24/30fps", "4K@30/60/120fps"]
  },
  "battery": {
    "capacity_mah": 5000,
    "charging_wired_w": 45,
    "charging_wireless_w": 15
  }
}
```

Indexing Strategy:
Using EF Core 10, we can create indices on specific paths within this JSON.

- *Scenario:* A user wants phones with > 108MP cameras.
- *Index:* CREATE INDEX idx_camera_mp ON Catalog.Devices ((Specifications->'main_camera'->'modules'->0->>'mp')::int);

---

4. The Multi-Vendor Marketplace Ecosystem

GSMSharing's request explicitly includes a "marketplace for electronics." This moves the platform beyond a passive wiki into a complex transactional engine. The schema must support **Multi-Vendor** logic (like Amazon or eBay), where one order can contain items from multiple sellers, requiring complex shipping and commission calculations.[10]

4.1 Vendor Identity and Profiles

A user can be a buyer, but upgrading to a "Vendor" requires additional data (KYC, payout details).

Table: Commerce.Vendors

Column	Type	Description
Id	INT	PK.
UserId	UUID	FK to Identity.Users.
StoreName	NVARCHAR(100)	Display name of the shop.
Slug	VARCHAR(100)	URL for the store: gsmsharing.com/shop/mobile-fixer.
CommissionRate	DECIMAL(5,4)	e.g., 0.0500 (5%). Platform fee per sale.
IsVerified	BIT	Result of admin KYC check.
PayoutConfig	JSONB	Encrypted bank/PayPal details for payouts.

4.2 Inventory and Listings

We must distinguish between the *Catalog Device* (The specs) and the *Listing* (The actual unit for sale).

Table: Commerce.Listings

Column	Type	Description
Id	UUID	PK.
VendorId	INT	FK to Vendors.
CatalogDeviceId	UUID	Nullable FK. Links listing to specs if it's a phone.
Title	NVARCHAR(200)	"Samsung S24 Ultra - Broken Screen".
Condition	TINYINT	0=New, 1=Open Box, 2=Used, 3=For Parts.
Price	DECIMAL(19,4)	
StockQuantity	INT	Inventory tracking.
ShippingProfileId	INT	FK to shipping rules (weight/dimensions).
Images	JSONB	Array of actual product photos (not stock photos).

4.3 Order Orchestration (Split Payments)

When a customer buys items from two different vendors in one cart, the system must create a "Master Order" for the customer and multiple "Sub Orders" for the vendors.

Table: Commerce.Orders (Master)

Column	Type	Description
Id	UUID	PK. Customer-facing Order ID.
CustomerId	UUID	FK to Identity.Users.

TotalAmount	DECIMAL	Grand total paid.
PaymentGatewayRef	VARCHAR(100)	Stripe/PayPal Transaction ID.
CreatedAt	DATETIME	
PaymentStatus	TINYINT	Pending, Paid, Failed, Refunded.

Table: Commerce.SubOrders (Vendor Slice)

Column	Type	Description
Id	UUID	PK. Vendor-facing Order ID.
MasterOrderId	UUID	FK to Orders.
VendorId	INT	FK to Vendors.
SubTotal	DECIMAL	Amount for this vendor's items.
CommissionFee	DECIMAL	Amount retained by GSMSharing.
PayoutStatus	TINYINT	Scheduled, Paid, Held.
ShippingStatus	TINYINT	Processing, Shipped, Delivered.
TrackingNumber	VARCHAR(100)	
Carrier	VARCHAR(50)	FedEx, DHL, USPS.

Table: Commerce.OrderItems

Column	Type	Description
Id	UUID	PK.
SubOrderId	UUID	FK to SubOrders.
ListingId	UUID	FK to Listings.
SnapshotPrice	DECIMAL	Price at moment of purchase (in case listing changes).
Quantity	INT	

---

5. Community, Identity, and Gamification

The user request emphasizes "forums," "user profiles," and "voting." The forum component is likely replacing a legacy phpBB/vBulletin system. The new schema must support modern, real-time interactions and a gamified reputation system to drive engagement.[12]

5.1 ASP.NET Core Identity Extension

We utilize the robust Microsoft.AspNetCore.Identity system but extend it to avoid bloating the core authentication table.

Table: Identity.AspNetUsers (Standard)

- Id, UserName, Email, PasswordHash, TwoFactorEnabled.

Table: Identity.UserProfiles (Community Data)

Column	Type	Description
UserId	UUID	PK, FK to AspNetUsers.
DisplayName	NVARCHAR(50)	Public name (different from login username).
AvatarUrl	VARCHAR(500)	

Signature	NVARCHAR(1000)	Legacy forum signature (BBCode/HTML).
ReputationPoints	INT	Cached sum of all reputation logs.
PostCount	INT	Cached count for forum ranks.
TrustLevel	TINYINT	0=New, 1=Member, 2=Verified, 3=Mod.

5.2 Forum Hierarchical Schema

Table: Community.Categories

- Id, Name, ParentId (Self-referencing FK for nested sub-forums), SortOrder, IsActive.

Table: Community.Topics

Column	Type	Description
Id	UUID	PK.
CategoryId	INT	FK to Categories.
AuthorUserId	UUID	FK to Users.
Title	NVARCHAR(200)	
Type	TINYINT	0=Discussion, 1=Question, 2=Poll.
IsPinned	BIT	Sticky threads.
IsLocked	BIT	
ViewCount	BIGINT	

LastPostAt	DATETIME	For sorting "Recently Active".

Table: Community.Posts

Column	Type	Description
Id	UUID	PK.
TopicId	UUID	FK to Topics.
AuthorUserId	UUID	FK to Users.
Content	NVARCHAR(MAX)	The post body. Supports Markdown/BBCode.
ReplyToPostId	UUID	Nullable. Allows threaded view (Reddit-style) or flat view.
EditCount	INT	

5.3 Gamification Engine (Reputation & Voting)

To support "Voting" and "Top 5 Phones" lists [14], we need a generic voting mechanism that applies to multiple entity types (Phones, Posts, Reviews).

Table: Community.Votes

Column	Type	Description
Id	BIGINT	PK.
UserId	UUID	The voter.
TargetEntityId	UUID	ID of the Phone or Post being voted on.

TargetEntityType	VARCHAR(50)	'Device', 'Post', 'Listing'.
Value	SMALLINT	+1 (Upvote) or -1 (Downvote).
Timestamp	DATETIME	

Table: Community.ReputationLog
An immutable ledger tracking why a user has points.

Column	Type	Description
Id	BIGINT	PK.
UserId	UUID	The user receiving points.
EventCode	VARCHAR(50)	'DAILY_LOGIN', 'ACCEPTED_ANSWER', 'SOLD_ITEM'.
Points	INT	+5, +10, etc.
ReferenceId	UUID	Link to the post/order that triggered this.
CreatedAt	DATETIME	

---

6. Firmware Repository and File Management

A unique requirement of GSMSharing is the "Flashing files" repository.[4] Unlike generic e-commerce, this requires handling large binary assets (firmware images) which can be 5GB+ in size.

Storage Architecture Warning:
It is a critical anti-pattern to store large BLOBs (Binary Large Objects) directly in the SQL database. It bloats the transaction logs and destroys backup performance.15
Strategy: Store files in an S3-compatible Object Storage (AWS S3, MinIO, Azure Blob). The database stores the metadata and secure access tokens.

Table: Repository.Files

Column	Type	Description
Id	UUID	PK.
DeviceId	UUID	FK to Catalog.Devices.
Filename	VARCHAR(255)	"G998BXXU1AUB6_Odin.zip".
VersionCode	VARCHAR(50)	"Android 14 - OneUI 6.1".
RegionCode	VARCHAR(10)	"XEU", "DBT", "VZW".
FileSize	BIGINT	Bytes. Used for "Download Progress" estimation.
StoragePath	VARCHAR(500)	"s3://firmware-bucket/samsung/s21/v14.zip".
ChecksumMD5	CHAR(32)	Essential for verifying firmware integrity before flashing.
IsPremium	BIT	If true, requires 'Premium' role to generate link.
UploadDate	DATETIME	

Table: Repository.Partitions
For advanced users, we can parse the firmware zip (using Python) and list the internal images.17

- Id, FileId, PartitionName ("boot.img", "recovery.img", "system.img"), Size.

---

7. Content Enhancement and Python Interop

The "Python for content enhancement" requirement suggests an asynchronous processing pipeline. Python libraries (like Pandas, NLTK, or PyTorch) are superior for data analysis tasks.

7.1 The Job Queue Schema

We avoid direct HTTP calls between ASP.NET and Python to prevent timeouts. We use a database-backed queue pattern.

Table: Sys.PythonJobs

Column	Type	Description
Id	BIGINT	PK.
JobType	VARCHAR(50)	'GENERATE_SUMMARY', 'FETCH_SPECS', 'ANALYZE_SENTIMENT'.
Payload	JSONB	Arguments: {"deviceId": "xyz", "url": "..."}.
Status	TINYINT	0=Pending, 1=Processing, 2=Completed, 3=Failed.
Result	JSONB	Output from Python script.
CreatedAt	DATETIME	
ProcessedAt	DATETIME	

7.2 Enhancement Workflows

1. **Sentiment Analysis:** When a user reviews a phone, a job is queued. Python analyzes the text, extracts keywords (e.g., "Overheating", "Great Camera"), and writes a JSON summary back to the Reviews table.
2. **Missing Spec Scraper:** If a device has null values for "Battery", a Python job uses BeautifulSoup to scrape trusted sources (manufacturer sites) and suggests updates.[5]

---

8. Affiliate Marketing and Analytics Schema

To monetize the content like GSMArena, an internal affiliate tracking system is required. This system tracks outgoing clicks to partners (Amazon, eBay) and attributes conversions.[18]

Table: Marketing.AffiliatePartners

- Id, Name ("Amazon"), BaseUrl, TrackingTag ("gsmsharing-20").

Table: Marketing.AffiliateLinks
Maps a catalog device to a specific buy link.

- Id, PartnerId, DeviceId, TargetUrl, IsActive.

Table: Marketing.ClickEvents
Performance Note: This table will grow by millions of rows. In PostgreSQL, this should be a Partitioned Table (partitioned by CreatedAt month).

Column	Type	Description
Id	BIGINT	PK.
LinkId	INT	FK to AffiliateLinks.
SessionId	VARCHAR(100)	Anonymous user tracking.
Referrer	VARCHAR(500)	Where did they come from? (Google, Forum).
UserAgent	VARCHAR(255)	Browser/OS data.
IPAddress	INET	For fraud detection.
CreatedAt	DATETIME	

---

9. SEO and Infrastructure Strategy

9.1 SEO Optimization Schema

High ranking requires dynamic meta tags and structured data (Schema.org).[20]

Table: Content.SeoMetaData
This table uses a Polymorphic association to attach SEO tags to any entity (Device, Article, Vendor).

Column	Type	Description
Id	INT	PK.
EntityId	UUID	ID of the target object.
EntityType	VARCHAR(50)	'Device', 'Article'.
TitleTag	NVARCHAR(150)	<title> override.
MetaDescription	NVARCHAR(300)	<meta name="description">.
OpenGraphImage	VARCHAR(500)	Image for social sharing.
CanonicalUrl	VARCHAR(500)	Prevents duplicate content penalties.

Table: Content.UrlRedirects
Essential for migration. As we move from the old script (which might use IDs like /product.php?id=123) to new routes (/devices/samsung-s24), we need a persistent redirect map.22

- SourcePath (Indexed), TargetUrl, StatusCode (301/302).

9.2 Caching Strategy (Redis)

Database queries for the "Phone Finder" or "Menu" are expensive.

- **Redis Key:** device:{slug}:full_specs
- **Content:** Protobuf or JSON serialized Device object.
- **Expiration:** 1 hour or until Admin update.

---

10. Conclusion

The transformation of GSMSharing.com from a legacy script-based site to a modern ASP.NET Core 10 platform requires a database schema that is both rigid in its integrity and flexible in its data modeling. The proposed architecture achieves this by:

1. **Hybridizing Data:** Using Relational tables for Identity/Commerce and JSONB for Specifications, solving the "GSMArena granularity" problem without EAV performance

costs.
2. **Isolating Domains:** The modular schema design prevents the Forum's read load from locking the Marketplace's write transactions.
3. **Integrating Intelligence:** The dedicated Python Job Queue schema allows for advanced AI/ML features (Sentiment Analysis, Spec Prediction) to run asynchronously, enhancing the content value without impacting page load speeds.
4. **Securing Commerce:** The detailed Multi-Vendor schema ensures that split orders, commissions, and vendor payouts are handled with financial-grade precision.

This blueprint provides the "exhaustive detail" and "nuanced understanding" required to build a platform that not only matches the informational depth of GSMArena but exceeds it by layering a vibrant, transactional community marketplace on top. The inclusion of legacy migration paths ensures that the historical value of gsmsharing.com is preserved while its future scalability is secured.

Works cited

1. Tools - GSMArena.com, accessed December 15, 2025, https://www.gsmarena.com/tools.php3
2. GSMArena Comprehensive Mobile Phone Dataset - Kaggle, accessed December 15, 2025, https://www.kaggle.com/datasets/madajaswanth/phone-dataset-8461-phones
3. Phone Finder - search for a phone by feature - GSMArena.com, accessed December 15, 2025, https://www.gsmarena.com/search.php3
4. gsmsharing, accessed December 15, 2025, https://www.gsmsharing.com
5. Gsmarena.com Web Scraper - WebAutomation, accessed December 15, 2025, https://webautomation.io/pde/gsmarenacom-extractor/351/
6. Storing Dynamic Attributes - Sparse Columns, EAV, and JSONB Explained | Leapcell, accessed December 15, 2025, https://leapcell.io/blog/storing-dynamic-attributes-sparse-columns-eav-and-jsonb-explained
7. PostgreSQL JSONB vs. EAV: Which is Better for Storing Dynamic Data? - Raz Samuel Blog -, accessed December 15, 2025, https://www.razsamuel.com/postgresql-jsonb-vs-eav-dynamic-data/
8. Deep JSON Integration: New Capabilities in EF Core 9 | by Ramazan Gunes - Medium, accessed December 15, 2025, https://gunesramazan.medium.com/deep-json-integration-new-capabilities-in-ef-core-9-a7e288983987
9. JSON Columns in Entity Framework Core, accessed December 15, 2025, https://www.learnentityframeworkcore.com/misc/json-columns
10. Ultimate guide to payments in multi-vendor marketplaces - Knowledge Source, accessed December 15, 2025, https://knowledge.antom.com/ultimate-guide-to-multi-vendor-marketplaces
11. How to Build a Multi-Vendor eCommerce Marketplace - Aalpha Information Systems, accessed December 15, 2025,

https://www.aalpha.net/blog/how-to-develop-a-multi-vendor-ecommerce-marketplace/
12. Is there a howto on gamification from a developer's point of view? - Reddit, accessed December 15, 2025, https://www.reddit.com/r/gamification/comments/1pjmmz8/is_there_a_howto_on_gamification_from_a/
13. 10 Examples of Badges Used in Gamification - Trophy, accessed December 15, 2025, https://trophy.so/blog/badges-feature-gamification-examples
14. Frequently Asked Questions - GSMArena.com, accessed December 15, 2025, https://www.gsmarena.com/faq.php3
15. Sending/Storing big data with Asp.Net and EF Core : r/csharp - Reddit, accessed December 15, 2025, https://www.reddit.com/r/csharp/comments/snwfd5/sendingstoring_big_data_with_aspnet_and_ef_core/
16. Strategies for Handling Large File Uploads in a .NET Web API? - Microsoft Learn, accessed December 15, 2025, https://learn.microsoft.com/en-us/answers/questions/1375143/strategies-for-handling-large-file-uploads-in-a-ne
17. Some information about firmware files - GitHub Gist, accessed December 15, 2025, https://gist.github.com/lopestom/ce250f5de64a2764ee85092a2c01939e
18. Affiliate Marketing Tracker Database Structure and Schema, accessed December 15, 2025, https://databasesample.com/database/affiliate-marketing-tracker-database
19. What Is Affiliate Conversion Tracking: A Guide - Tapfiliate, accessed December 15, 2025, https://tapfiliate.com/blog/affiliate-conversion-tracking/
20. Programmatically Injecting SEO Friendly Tags In ASP.NET Core - C# Corner, accessed December 15, 2025, https://www.c-sharpcorner.com/article/programmatically-injection-of-seo-friendly-tags-in-asp-net-core/
21. Getting started with schema.org using Microdata, accessed December 15, 2025, https://schema.org/docs/gs.html
22. Routing - Astro Docs, accessed December 15, 2025, https://docs.astro.build/en/guides/routing/